

Trabalho Prático 1 — Implementação de um Controlador de *Cache* em SystemVerilog

Antônio D. C Sousa, Davi F. Puddo, Gabriel V. B. Silva,
Mateus H. M. Diniz, Raquel P. Motta

¹Graduação em Ciência da Computação
Instituto de Ciências Exatas e Tecnologia, PUC Minas
Caixa Postal, 1.686 – Belo Horizonte – MG – Brazil
Minas Gerais – MG – Brasil

{adcsousa, davi.puddo, gabriel.silva.1524976}@sga.pucminas.br

{mateus.diniz.880541, raquel.motta}@sga.pucminas.br

Resumo. *Este relatório descreve a implementação de um controlador de cache em SystemVerilog, baseado no Capítulo 5, Seção 5.12 do livro Computer Organization and Design: The Hardware/Software Interface — RISC-V Edition, de Patterson e Hennessy. O projeto consiste em uma cache de mapeamento direto, com 1024 linhas de 128 bits, política write-back com write-allocate, descrita como uma máquina de estados finitos. São discutidas as decisões de projeto, as dificuldades encontradas, a estrutura do testbench usado para validação funcional e o papel das ferramentas de IA no processo.*

Abstract. *This report describes the implementation of a cache controller in SystemVerilog, based on Chapter 5, Section 5.12 of Computer Organization and Design: The Hardware/Software Interface — RISC-V Edition by Patterson and Hennessy. The design is a direct-mapped cache with 1024 lines of 128 bits, using a write-back / write-allocate policy and structured as a finite-state machine. We discuss design decisions, issues encountered, the structure of the testbench used for functional validation, and the role of AI tools throughout the process.*

1. Introdução

Caches reduzem a diferença de latência entre CPU e memória principal explorando localidade temporal e espacial. Este trabalho implementa o controlador de *cache* descrito na Seção 5.12 de [Patterson and Hennessy 2021], em SystemVerilog. Trata-se de uma *cache blocking*, de mapeamento direto, com 1024 blocos de 128 *bits*, política *write-back* com *write-allocate* e um bit `valid` e um bit `dirty` por linha. A Figura 1 reproduz o diagrama em blocos do livro, com os nomes dos sinais usados na descrição em SystemVerilog, que serviu de referência principal para a implementação.

A proposta envolveu usar os exemplos de código fornecidos no livro e seu material complementar como base para codificar um simulador compilável e funcional.

1.1. Visão geral da arquitetura

O projeto está em `src/cache_def.sv` e contém três módulos: `dm_cache_data` (memória de dados, 1024 linhas de 128 *bits*), `dm_cache_tag` (memória de *tags*, mesma

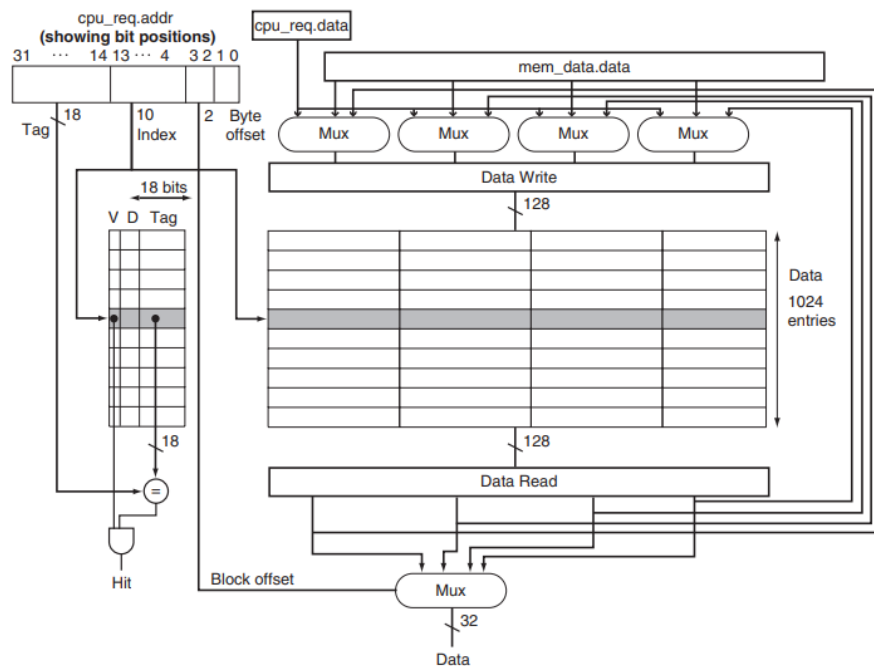


FIGURE e5.12.3 Block diagram of the simple cache using the Verilog names. Not shown are the write enables for the cache tag memory and for the cache data memory, or the control signals for multiplexors that supply data for the Data Write variable. Rather than have separate write enables on every word of the cache data block, the Verilog reads the old value of the block into Data Write and then updates the word in that variable on a write. It then writes the whole 128-bit block.

Figura 1. Diagrama em blocos da *cache* usando os nomes do SystemVerilog, conforme Figura e5.12.3 do livro [Patterson and Hennessy 2021].

profundidade, com *valid*, *dirty* e o vetor de *tag*) e *dm_cache_fsm* (contém toda a lógica de controle). As memórias têm escrita síncrona com *write enable* e leitura combinacional, exatamente como em [Patterson and Hennessy 2021].

O endereço de 32 *bits* é decomposto em *tag* (*addr*[31:12]), índice (*addr*[11:2]) e seletor de palavra dentro da linha (*addr*[1:0]), de modo a indexar 4 palavras de 32 *bits* por linha. Os bits da *tag* são referenciados pelos parâmetros TAGMSB e TAGLSB.

31	12	11	2	1	0
Tag		Índice da linha		Byte Offset	
20 bits		10 bits		2 bits	

A CPU e a memória principal não fazem parte do controlador propriamente dito: são simuladas a partir do *testbench*. Para a CPU, o arquivo de *testbench* simplesmente envia requisições de leitura/escrita *hard-coded*. Para a memória, foi utilizado um arranjo associativo que funciona de modo semelhante a um dicionário ou tabela hash, e gera dados aleatórios para popular endereços acessados pela primeira vez.

2. Disponibilidade do projeto

A implementação do controlador de cache foi realizada em Verilog e encontra-se publicamente disponível em <https://github.com/davipuddo/Arq3-TP1>, junto à implementação dos testes de validação.

3. Desenvolvimento

O código apresentado no livro mistura SystemVerilog com convenções tipográficas próprias e contém pequenas incorreções (espaçamento ausente entre tipo e identificador, uso de aspas invertidas no lugar de apóstrofo em literais '0, falta de begin/end em alguns ramos do case). A primeira etapa consistiu em transcrever as linhas para um único arquivo .sv e ajustar essas construções até obter compilação limpa, garantindo também que todos os sinais do bloco `always_comb` fossem inicializados com valores *default*, evitando inferência de *latches*.

3.1. Máquina de estados

Após a adaptação inicial do código, o próximo passo foi estudar e mapear o comportamento da FSM (Máquina de Estados Finitos) que rege o funcionamento da cache, que contém quatro estados: `idle`, `compare_tag`, `allocate` e `write_back`. O diagrama que produzimos está ilustrado na Figura 2.

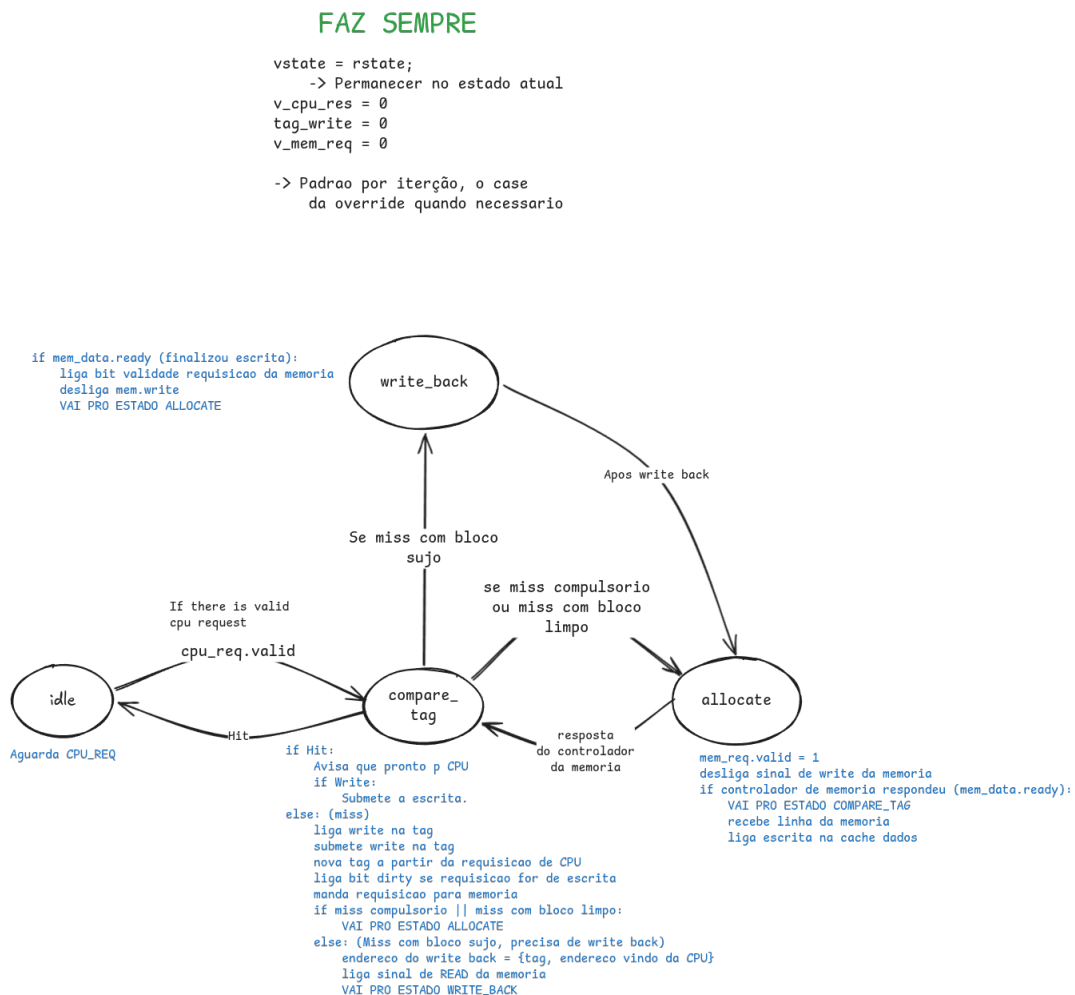


Figura 2. Máquina de estados do controlador de *cache*, com transições, condições e ações associadas a cada estado.

No estado `idle`, a FSM apenas espera por uma requisição da CPU: quando `cpu_req.valid = 1`, ocorre a transição para `compare_tag`. Nesse segundo estado,

comparam-se a *tag* lida da memória de *tags* com a *tag* do endereço pedido, levando em conta também o bit *valid* da entrada. Se houver *hit*, a transação se resolve em um único ciclo — `cpu_res.ready = 1` e a FSM volta direto para *idle*. No caso particular de um *write hit*, a linha correspondente ainda é marcada como *dirty* antes do retorno.

Se houver *miss*, dois caminhos são possíveis: (1) quando o bloco atualmente alocado naquele índice é *clean* (dirty bit desativado indica que não modificamos os dados da cache e, portanto, não há necessidade de copiá-los para a memória) ou quando o bit *valid* está em zero (caracterizando um *miss* compulsório, isto é, os dados naquele espaço são descartáveis). Nesses casos, parte-se direto para *allocate*. (2) quando o bloco for *dirty*. É preciso primeiro escrevê-lo de volta na memória principal, e por isso a FSM passa por *write_back* antes de chegar em *allocate*.

No estado *allocate*, mantém-se uma requisição de leitura à memória principal até que ela responda com `mem_data.ready`; quando isso acontece, a linha da *cache* é atualizada e a FSM volta para *compare_tag*, e dessa vez o acesso necessariamente vai dar *hit*, porque a linha correta acabou de ser carregada. Já em *write_back* a lógica é mais simples: ficamos esperando o *ack* da memória e, assim que ele chega, submetemos imediatamente a requisição de leitura nova (para a linha que vai substituir a antiga) e seguimos para *allocate*.

3.2. Comunicação entre CPU, cache e memória

O controlador conversa em duas interfaces:

- Com a CPU, recebe *CPURequest* (endereço, dado, tipo de operação *rw* e um bit *valid*) e devolve *CPUResult* (dado lido e um bit *ready*).
- Com a memória, envia *MemRequest* (endereço, linha de 128 *bits*, *rw* e *valid*) e recebe *MemData* (linha lida e *ready*).

Em ambas, o bit *valid* cumpre o papel de *request strobe* (um strobe é um sinal de controle dedicado usado para sincronizar a transferência de dados entre dois componentes de hardware): do lado da CPU, indica que há um acesso a ser tratado; do lado do controlador, indica à memória que uma requisição está sendo submetida naquele ciclo.

A memória só toma o trabalho de responder quando `req.valid` estiver em 1. No estado *compare_tag*, por exemplo, o `v_mem_req.valid` só é levantado *quando o caminho de miss é percorrido*. Nos demais casos ele é deixado em zero pelo *default* do início do bloco combinacional. Isso foi uma dúvida do grupo, e o prompt usado para investigá-la está na seção sobre uso de IA.

Por fim, o bit *ready* sinaliza ao controlador que a memória já terminou de processar a requisição. Como `v_mem_req.valid` é zerado por *default* no início do *always_comb*, ele só sobe nos ramos da FSM em que de fato ocorre acesso à memória (caminho de *miss*), evitando requisições espúrias.

Do ponto de vista da modelagem, essas interfaces constituem uma abstração das conexões físicas entre módulos. Em uma implementação em *hardware*, a comunicação entre CPU, cache e memória se daria por meio de barramentos — conjuntos de sinais elétricos roteados entre os blocos. Na descrição em SystemVerilog, esses barramentos são representados por estruturas tipadas (*CPURequest*, *CPUResult*, *MemRequest* e

MemData) cujos campos agregam todos os sinais de uma direção do barramento em um único objeto, manipulado por nome no corpo dos módulos. As variáveis intermediárias `v_mem_req` e `v_cpu_res`, atualizadas no bloco `always_comb` e amarradas às portas de saída por meio de `assign`, exercem o papel de *buffers* de interface entre o estado interno da FSM e os módulos vizinhos, garantindo que cada ciclo apresente um valor bem definido em cada porta. A Figura 2 ilustra essa abstração.

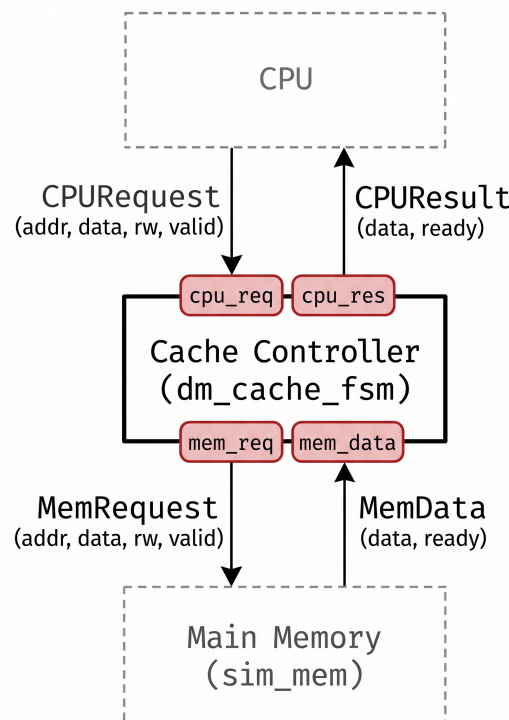


Figura 3. Comunicação entre módulos. As caixas vermelhas representam as estruturas tipadas que substituem os barramentos físicos do hardware.

3.3. Dificuldades com a memória simulada

A descrição original da memória simulada usava um arranjo associativo declarado como `bit [127:0] mem[*]`. Em SystemVerilog, esse `[*]` indica um *wildcard associative array*, em que a chave é qualquer valor inteiro.

O problema é que essa construção não é suportada de forma satisfatória pelo Verilator (a ferramenta usada neste trabalho), e algumas operações sobre o arranjo não funcionavam como o esperado, especialmente a operação de gravar dados no arranjo. O resultado era que a memória retornava 0 em todas as operações de leitura. Foi um dos pontos em que perdemos mais tempo de *debug*, porque a FSM parecia estar funcionando corretamente, mas todas as operações de leitura na memória tinham 0 como resultado, mesmo após gravar dados no mesmo endereço, quando na verdade o problema estava do outro lado da interface, na memória simulada.

A solução foi trocar o `[*]` por uma chave de tipo explícito, com largura coerente com o endereçamento que estamos usando:

```
bit [127:0] mem[reg[29:0]];
```

A escolha do uso de 30 bits e não 32 para endereçar a memória foi feita para invalidar endereços muito altos no ponto de vista da cache de forma a explorar o comportamento da cache e memória simuladas quando acesso for feito a endereços extremos.

3.4. Outros desafios e decisões de projeto

A nível arquitetural, seguimos a proposta do livro: mapeamento direto com 1024 linhas de 128 *bits*; política *write-back* (a linha só é gravada na memória ao ser substituída, controlada pelo bit *dirty*) combinada com *write-allocate* (em *write miss* o bloco é carregado antes da escrita); FSM Moore, com estado registrado e saídas calculadas em um único *always_comb*; e interfaces com *valid/ready* como *handshake*.

No nível do código, além da questão da memória simulada, outros pontos exigiram decisão própria. O primeiro foi de *legibilidade*: os *typedefs* originais (*cache_tag_type*, *cpu_req_type*, etc.) foram preservados por consistência com o livro, mas receberam *alias*es em *PascalCase* usados nas declarações de portas e sinais (*CPURequest* *cpu_req* lê mais naturalmente do que *cpu_req_type* *cpu_req*). A *struct* de *tag* mantém os campos nomeados (*valid*, *dirty*, *tag*), permitindo acesso por nome no corpo da FSM.

O segundo foi a *parametrização* de TAGMSB e TAGLSB. No livro, ambos são introduzidos como *parameter int* com valores 31 e 14. Mantivemos a parametrização, mas com TAGLSB = 12, refletindo a opção por usar *addr[1:0]* como seletor de palavra dentro da linha (em vez de *addr[3:2]* como no livro, que pressupõe endereçamento *byte-aligned*). Centralizar esses valores no pacote *cache_def*, em vez de espalhá-los como literais pelo RTL, isola a geometria da *cache* em um único ponto.

4. Testes de validação

4.1. Estrutura do *testbench*

O ambiente de simulação foi desenvolvido em um único arquivo contendo dois módulos principais: *sim_mem*, responsável por emular o comportamento e os atrasos da memória principal, e *test_main*, que atua como o *testbench* propriamente dito. O módulo principal instancia a FSM da cache, gerencia o clock e aplica um pulso de reset (*rst*), colocando o controlador em seu estado inicial. Vale ressaltar que, devido à tipagem bit do System-Verilog, todos os arrays de dados e tags são inicializados com 0 por padrão. Ademais, a sequência de instruções básicas necessárias para leitura e escrita foram encapsuladas em *read* e *write* respectivamente, além de padronizar as estruturas de validação de testes com exibição visual, para facilitar a compreensão e implementação.

```
task read(input bit[31:0] address);
    iu_req.rw = '0;
    iu_req.addr = address;
    iu_req.valid = '1;
    wait(iu_res.ready == '1);
endtask

task write(input bit[31:0] address, input bit[31:0] data);
    iu_req.rw = '1;
    iu_req.addr = address;
    iu_req.data = data;
```

```

        iu_req.valid = '1;
        wait(iu_res.ready == '1);
    endtask
    `define mksection(number, title) \
        $display("\033[35m%s\033[0m: \033[36m%s\033[0m", number, title
        );

    `define check_test(name, lhs, rhs, op) \
        if (lhs op rhs) begin \
            $display("\t\033[34mtest [%s]\033[0m: \033[32mok\033[0m",
                name); \
        end else begin \
            $display("\t\033[34mtest [%s]\033[0m: \033[31mfailed\033[0m
                ", name); \
        end \
        $display("\t\tCondition: \033[36m%s\033[0m (\033[33m%h\033[0m)
            \033[35m%s\033[0m \033[36m%s\033[0m (\033[33m%h\033[0m)",
            "lhs", lhs, "op", "rhs", rhs); \
        iu_req.valid = 0; \
    #5;

```

4.2. Etapas de teste

O testes foram divididos em cinco etapas: i) Testes de Leitura (Read Path); ii) Teste de Escrita (Write Path); iii) Testes de Substituição; iv) Testes de Consistência e v) Testes de Casos Limite. Que por sua vez foram subdivididos de acordo com as etapas necessárias para o objetivo.

4.2.1. Leitura

Nesse momento, o objetivo é validar o funcionamento correto das operações de leitura, nos casos de hit, com os dados requeridos já devidamente armazenados em cache, e com necessidade de acesso a memória principal, verificando posteriormente a atualização dos bits de controle: *tag* e *valid*.

Em relação ao hit, é necessário implementar duas leituras no mesmo endereço, onde a primeira resultaria em miss compulsório, tornando necessária a busca dos dados na memória principal como alocação inicial. Dessa forma, possibilita-se que a segunda realize o hit efetivo, aguardando um intervalo de tempo simulado entre ambas, validando por meio da análise do status da última requisição.

```

    `mksection("7.1.1", "Access with cache hit");
    read(0);
    iu_req.valid = 0;
    #5;
    read(0);
    prev_hit = iu_res.hit;
    `check_test("checking hit on read", prev_hit, 1, ==);

```

Para garantir a não ocorrência de um falso hit (quando a cache ignora a mudança de tag), simulou-se um acesso à memória principal forçando uma substituição de bloco. Foram realizadas operações de escrita nos endereços 0x00000000 e 0x00001000, que,

por compartilharem os mesmos bits de índice, mapeiam para a mesma linha da cache. Posteriormente, eles são acessados para leitura na mesma sequência, momento em que são realizadas duas verificações: se a segunda leitura acarretou em um miss e se os dados da primeira leitura diferem da segunda.

```
`mksection("7.1.2", "Access with cache hit followed by memory
load");
write('h00000000, 'hF00DCAFE);
write('h00001000, 'hCAFEBABE);
read('h00000000);
prev_data = iu_res.data;
#5;
read('h00001000);
`check_test(
    "checking miss data change on read",
    prev_data, iu_res.data, !=
);
`check_test(
    "checking miss on read",
    iu_res.hit, 0, ==
);
```

A fim de validar os bits de controle *tag* e *valid*, foram realizadas duas leituras no endereço 0xCAFFE000 a primeira deve ocasionar um miss com *iu_res.line_tag.valid = 0* e assim trazer os dados com a respectiva tag 0xCAFFE da memória, uma segunda leitura ao mesmo endereço visa constatar que o bit de validade agora indica que o endereço desejado já está presente na cache.

```
`mksection("7.1.3", "Tag & valid bit values");
read('hCAFFE000);
`check_test(
    "checking tag value after allocation",
    iu_res.line_tag.tag, 'hCAFFE, ==
);
`check_test(
    "checking valid bit value after allocation",
    iu_res.line_tag.valid, '0, ==
);
read('hCAFFE000);
`check_test(
    "checking valid bit value on hit",
    iu_res.line_tag.valid, '1, ==
);
```

4.2.2. Escrita

Para a avaliação do caminho de dados de escrita, foram simuladas três situações: escrita com *hit*, escrita com *miss* e validação do bit de controle de modificação (*dirty*) referente à política de escrita. A primeira validação foi implementada realizando operações de leitura e escrita de forma subsequente sobre o mesmo endereço. A leitura inicial atua como um *miss* compulsório, garantindo que o bloco seja alocado na *cache*. Dessa forma, a operação de escrita logo em seguida resulta obrigatoriamente em um *hit*. Por fim, uma nova leitura

é realizada para constatar se os dados foram devidamente atualizados e armazenados na *cache* interna.

```
\mksection("7.2.1", "Write with hit");
read('h00002000);
\check_test("allocating block for write", iu_res.hit, 0, ==);
write('h00002000, 'hDEADBEEF);
\check_test("checking write hit", iu_res.hit, 1, ==);
read('h00002000);
\check_test("checking data written on hit", iu_res.data, '
    hDEADBEEF, ==);
```

Posteriormente, para o teste de escrita com *miss* e para a comprovação da política de *Write-Allocate* (alocação na escrita), a *cache* recebeu uma instrução de escrita direta em um endereço arbitrário que ainda não havia sido acessado. Espera-se que essa operação acarrete um *miss*, forçando o controlador a buscar o bloco na memória principal antes de aplicar a modificação. O sucesso da alocação e da escrita é validado por uma leitura subsequente, submetendo o endereço à mesma análise anterior para atestar se o dado retornado é exatamente o valor recém-escrito.

```
\mksection("7.2.2", "Write with miss");
write('h00003000, 'hBEEFCAFE);
\check_test("checking write miss", iu_res.hit, 0, ==);
read('h00003000);
\check_test("checking data written on miss", iu_res.data, '
    hBEEFCAFE, ==);
```

A seguir, a simulação foi direcionada à validação da política de *Write-Back* (escrita postergada). Para isso, realizou-se a escrita em um novo endereço arbitrário, seguida imediatamente por uma leitura de inspeção. O objetivo dessa leitura posterior é revelar o estado dos metadados da *cache*, exigindo que o controlador tenha ativado corretamente o bit *dirty* (*iu_res.line_tag.dirty == 1*), sinalizando que o bloco sofreu modificação local e está inconsistente em relação à memória principal.

```
\mksection("7.2.3", "Write policy");
write('h00004000, 'h12345678);
\check_test("allocating for policy check", iu_res.hit, 0, ==);
read('h00004000);
\check_test("checking dirty bit on write", iu_res.line_tag.
    dirty, 1, ==);
```

4.2.3. Substituição

A simulação para as políticas de substituição foi realizada visando avaliar tanto a substituição simples (de blocos não modificados) quanto a substituição de dados em estado *dirty*, que demandam a utilização do mecanismo de *write-back*. A princípio, foram realizadas duas leituras em endereços diferentes que mapeiam para o mesmo índice na *cache*, provocando uma substituição de bloco. Uma terceira leitura, realizada novamente sobre o primeiro endereço, teve como objetivo verificar se a evicção ocorreu como o esperado. A análise do bit *hit* atesta que todas as três operações levaram a um *miss*, comprovando o correto funcionamento do mapeamento direto.

```

`mksection("7.3.1", "Cache fill & block substitution");
read('h00005000);
`check_test("checking fill miss", iu_res.hit, 0, ==);
read('h00006000);
`check_test("checking substitution miss", iu_res.hit, 0, ==);
`mksection("7.3.2", "Cache substitution policy");
read('h00005000);
prev_hit = iu_res.hit;
`check_test("checking direct mapped eviction", iu_res.hit, 0,
    ==);

```

Para validar a funcionalidade de *write-back*, simulou-se a criação de um bloco *dirty* através de uma operação de escrita no endereço 0x00007000. Posteriormente, uma leitura no endereço 0x00008000, que mapeia para o mesmo índice da escrita anterior, forçou uma nova substituição. Devido ao estado sujo da linha, tornou-se necessária a reescrita prévia do bloco antigo na memória principal antes da alocação do novo. Por fim, para confirmar a correta transição de estados do controlador, observou-se o sinal *iu_res.write_back*, cujo valor em nível lógico alto atesta a aplicação rigorosa da política.

```

`mksection("7.3.3", "Write-back");
write('h00007000, 'hAABBCCDD);
`check_test("write allocating 7000", iu_res.hit, 0, ==);
read('h00008000);
`check_test("checking write-back occurred", iu_res.write_back,
    1, ==);

```

4.2.4. Consistência

Para avaliar a consistência do controlador, foram implementadas três simulações distintas. A primeira verifica a coerência de dados, realizando a escrita em um endereço seguida imediatamente por uma leitura, a fim de validar se os dados recém-modificados foram efetivamente armazenados e disponibilizados. A segunda analisa o comportamento da máquina de estados diante de acessos de leitura sequenciais a um mesmo endereço, observando a ocorrência de um *miss* compulsório inicial seguido de *hits* ininterruptos. Por fim, a terceira simulação aborda cenários de conflito, alternando acessos de leitura a diferentes endereços que mapeiam para o mesmo índice na *cache*. A validação ocorre através da análise do bit de *hit*, confirmando que cada requisição alternada resulta em um novo *miss* devido à evicção contínua dos blocos.

```

`mksection("7.4.1", "Data coherence");
write('h00009000, 'h99999999);
`check_test("write allocating 9000", iu_res.hit, 0, ==);
read('h00009000);
`check_test("checking data coherence read after write", iu_res
    .data, 'h99999999, ==);
`mksection("7.4.2", "Repeated address access");
read('h0000A000);
`check_test("first read miss", iu_res.hit, 0, ==);
read('h0000A000);
`check_test("checking hit on repeated access", iu_res.hit, 1,
    ==);

```

```

read('h0000A000);
`check_test("checking hit on 3rd access", iu_res.hit, 1, ==);
`mksection("7.4.3", "Conflicts");
read('h0000B000);
`check_test("first read B000", iu_res.hit, 0, ==);
read('h0000C000);
`check_test("first read C000", iu_res.hit, 0, ==);
read('h0000B000);
`check_test("checking conflict miss", iu_res.hit, 0, ==);

```

4.2.5. Casos limite

Para finalizar a validação do controlador, a última etapa de testes avalia o comportamento da *cache* em situações extremas e verifica a integridade de seu estado inicial. A primeira simulação analisa o acesso a endereços nos limites absolutos do barramento de 32 bits (como 0xFFFFFFFF e 0xFFFFFFFF0), atestando que o roteamento de bits e o alinhamento de blocos operam corretamente sem causar falhas ou *overflows* nos registradores. A segunda abordagem foca na inicialização tardia, realizando a leitura de um endereço arbitrário virgem para garantir que o estado padrão da *cache* (linhas invalidadas) não sofreu corrupção durante o estresse das dezenas de operações e evicções anteriores. Por fim, a terceira simulação expande essa verificação ao executar leituras consecutivas em múltiplos blocos intocados, assegurando a robustez do isolamento da *cache* e a ausência de vazamento de metadados entre linhas vizinhas. Em todos os cenários, a validação é confirmada pela análise do bit de *hit*, exigindo rigorosamente que todas as requisições resultem em *misses*.

```

`mksection("7.5.1", "Edge case access");
read('hFFFFFFFF);
`check_test("checking edge case miss FFFFFFFF", iu_res.hit, 0, ==);
read('hFFFFFFFF0);
`check_test("checking edge case FFFFFFFF0", iu_res.hit, 0, ==);
`mksection("7.5.2", "Cache initialization");
read('h0000D000);
`check_test("checking initialization miss on untouched address", iu_res.hit, 0, ==);
`mksection("7.5.3", "Fully invalid cache behavior");
read('h0000E000);
`check_test("checking miss E000", iu_res.hit, 0, ==);
read('h0000F000);
`check_test("checking miss F000", iu_res.hit, 0, ==);

```

4.3. Resultados

Os resultados obtidos na simulação foram plenamente coerentes com o comportamento esperado em cada bateria de testes. O registro de execução atesta o sucesso da implementação do controlador de *cache*, validando a eficácia de suas políticas de alocação e substituição, bem como a sua estabilidade diante de casos extremos e cenários de estresse que poderiam induzir falhas no sistema. Conforme o relatório de saída gerado pelo simulador Verilator, todas as asserções lógicas foram cumpridas sem apresentar erros, finalizando a verificação com êxito.

```

7.1.1: Access with cache hit
    test [checking hit on read]: ok
        Condition: prev_hit (1) == 1 (000000001)
7.1.2: Access with cache hit followed by memory load
    test [checking miss data change on read]: ok
        Condition: prev_data (409b89cf) != iu_res.data (3dfa6bf3)
    test [checking miss on read]: ok
        Condition: iu_res.hit (0) == 0 (000000000)
7.1.3: Tag & valid bit values
    test [checking tag value after allocation]: ok
        Condition: iu_res.line_tag.tag (caffe) == 'hCAFFE (000caffe
    )
    test [checking valid bit value after allocation]: ok
        Condition: iu_res.line_tag.valid (0) == '0 (0)
    test [checking valid bit value on hit]: ok
        Condition: iu_res.line_tag.valid (1) == '1 (1)
7.2.1: Write with hit
    test [allocating block for write]: ok
        Condition: iu_res.hit (0) == 0 (000000000)
    test [checking write hit]: ok
        Condition: iu_res.hit (1) == 1 (000000001)
    test [checking data written on hit]: ok
        Condition: iu_res.data (deadbeef) == 'hDEADBEEF (deadbeef)
7.2.2: Write with miss
    test [checking write miss]: ok
        Condition: iu_res.hit (0) == 0 (000000000)
    test [checking data written on miss]: ok
        Condition: iu_res.data (beefcafe) == 'hBEEFCAFE (beefcafe)
7.2.3: Write policy
    test [allocating for policy check]: ok
        Condition: iu_res.hit (0) == 0 (000000000)
    test [checking dirty bit on write]: ok
        Condition: iu_res.line_tag.dirty (1) == 1 (000000001)
7.3.1: Cache fill & block substitution
    test [checking fill miss]: ok
        Condition: iu_res.hit (0) == 0 (000000000)
    test [checking substitution miss]: ok
        Condition: iu_res.hit (0) == 0 (000000000)
7.3.2: Cache substitution policy
    test [checking direct mapped eviction]: ok
        Condition: iu_res.hit (0) == 0 (000000000)
7.3.3: Write-back
    test [write allocating 7000]: ok
        Condition: iu_res.hit (0) == 0 (000000000)
    test [checking write-back occurred]: ok
        Condition: iu_res.write_back (1) == 1 (000000001)
7.4.1: Data coherence
    test [write allocating 9000]: ok
        Condition: iu_res.hit (0) == 0 (000000000)
    test [checking data coherence read after write]: ok
        Condition: iu_res.data (99999999) == 'h99999999 (99999999)
7.4.2: Repeated address access
    test [first read miss]: ok
        Condition: iu_res.hit (0) == 0 (000000000)
    test [checking hit on repeated access]: ok

```

```

        Condition: iu_res.hit (1) == 1 (00000001)
test [checking hit on 3rd access]: ok
        Condition: iu_res.hit (1) == 1 (00000001)
7.4.3: Conflicts
test [first read B000]: ok
        Condition: iu_res.hit (0) == 0 (00000000)
test [first read C000]: ok
        Condition: iu_res.hit (0) == 0 (00000000)
test [checking conflict miss]: ok
        Condition: iu_res.hit (0) == 0 (00000000)
7.5.1: Edge case access
test [checking edge case miss FFFFFFFF]: ok
        Condition: iu_res.hit (0) == 0 (00000000)
test [checking edge case FFFFFFFF0]: ok
        Condition: iu_res.hit (0) == 0 (00000000)
7.5.2: Cache initialization
test [checking initialization miss on untouched address]: ok
        Condition: iu_res.hit (0) == 0 (00000000)
7.5.3: Fully invalid cache behavior
test [checking miss E000]: ok
        Condition: iu_res.hit (0) == 0 (00000000)
test [checking miss F000]: ok
        Condition: iu_res.hit (0) == 0 (00000000)
- src/testbench.sv:296: Verilog $finish
- Simulation Report: Verilator 5.048 2026-04-26
- Verilator: $finish at 11us; walltime 0.002 s; speed 6.021 ms/s
- Verilator: cpu 0.002 s on 1 threads; allocated 7 MB

```

4.4. Discussão

A simulação executada até aqui se comporta de forma compatível com o que se espera de uma *cache write-back* de mapeamento direto: *hits* se resolvem em poucos ciclos, *misses* pagam o custo do MEM_DELAY configurado na memória simulada, e blocos sujos só são escritos na memória principal quando precisam ser substituídos. Não foram observadas inconsistências entre o dado retornado à CPU e o dado mantido na memória principal nos cenários executados.

5. Conclusão

A implementação do controlador de *cache* mostrou-se um exercício bastante interessante de tradução entre uma descrição de arquitetura — razoavelmente abstrata, no livro — e uma descrição de *hardware* sintetizável, com todos os pequenos detalhes de SystemVerilog. A maior parte do trabalho não esteve na *lógica* em si (a FSM é pequena e bem documentada no livro), mas em entender o *porquê* de cada sinal estar onde está, e em fazer com que o código realmente compilasse e simulasse no Verilator.

Entre as lições principais, destacamos:

- a importância de pensar na *cache* como uma máquina de estados, em vez de tentar tratá-la como um conjunto de casos independentes;
- o papel do bit *valid* (e do seu par *ready*) como mecanismo simples de *handshake* entre módulos;

- a necessidade de prestar atenção em construções específicas do SystemVerilog que dependem fortemente do simulador escolhido — como ocorreu com o `mem[*]` na memória simulada;
- o ganho de legibilidade obtido com *aliases* de tipos e `parameters` nomeados, ainda que sejam mudanças cosméticas em relação ao código original do livro.

Os próximos passos planejados envolvem reescrever parte do *testbench* para cobrir os cenários mínimos descritos no enunciado, incluir evidências visuais (waveforms) no relatório e revisar a sequência de operações *hard-coded* para garantir cobertura mais ampla dos caminhos da FSM.

6. Uso de Inteligência Artificial

Durante o desenvolvimento deste trabalho, ferramentas de IA (modelos de linguagem) foram utilizadas como apoio, especificamente em duas frentes: (i) **explicação** de trechos do código do livro cujo comportamento não estava claro à primeira leitura e (ii) **depuração** de problemas pontuais do SystemVerilog que envolviam construções específicas do simulador. Em nenhum momento a IA foi usada para gerar a implementação do controlador como um todo — a estrutura geral parte diretamente da descrição do livro, e as decisões de projeto foram tomadas pelo grupo.

Os *prompts* efetivamente utilizados, registrados no arquivo `promptsUsadosDesenvolvimento.txt` dentro da pasta `relatorio/`, foram:

“I’m trying to understand the FSM. In the compare_tag state, I have one doubt: If I understand correctly, setting the valid bit on a request means ‘submitting’ that request. Why, right before checking the compulsory miss, does it submit a memory request, before writing any information to v_mem_req ?”

“Alright, so this means that on the end of the compare_tag case statement is where the memory will actually read a request, right? But doesnt this means that, if no write back is needed, no other information will be written to the mem_req before going to the allocate state?”

Em todos os casos, as respostas da IA foram tratadas como *hipóteses* a serem verificadas, e não como verdades.

Referências

- ChipVerify. Rtl synthesis tutorial. <https://chipverify.com/tutorials/rtl-synthesis>. Acesso em: 20 maio 2026.
- ChipVerify. Systemverilog tutorial. <https://chipverify.com/tutorials/systemverilog>. Acesso em: 20 maio 2026.
- Patterson, D. A. and Hennessy, J. L. (2021). *Computer Organization and Design: The Hardware/Software Interface — RISC-V Edition*. Morgan Kaufmann, 2nd edition.